MINI PROJECT REPORT

Student Record Management System Using Python and MySQL

Submitted by KESHAVA K

Course / Semester DBMS, BCA (Hons), 3rd Semester

College RV University

1. Abstract

The Student Record Management System is a desktop application developed using Python, Tkinter, and a MySQL database. It allows an authorized user to add, view, search, update, and delete student records through a graphical interface. Unlike an application that stores data only in temporary variables or local files, this project stores records in a MySQL table so that the information remains available after the application is closed.

2. Objectives

• To design a simple and user-friendly student record application.

• To create and use a relational MySQL database and table.

• To demonstrate INSERT, SELECT, UPDATE, and DELETE operations.

• To validate user input and handle duplicate email addresses and database errors.

• To verify that changes made in the application are reflected in MySQL.

3. Tools and Technologies

Component	Technology / Purpose
Programming language	Python 3
Graphical user interface	Tkinter and ttk
Database	MySQL Server
Database connector	mysql-connector-python
Development environment	Visual Studio Code / MySQL Workbench

4. Database Design

Database name: **student_db**. Table name: **students**. The student_id field is an AUTO_INCREMENT primary key. The email field is unique to prevent duplicate student email addresses. Name, email, course, and semester are required; phone is optional. A semester value from 1 to 8 is accepted.

Table fields

Field	Data type	Constraint / use
student_id	INT	Primary key, auto increment
name	VARCHAR(80)	Required
email	VARCHAR(120)	Required, unique
course	VARCHAR(60)	Required
semester	TINYINT UNSIGNED	Required, 1–8
phone	VARCHAR(15)	Optional

created_at	TIMESTAMP	Default current timestamp

5. Application Modules

• **Add Student:** validates the form and inserts the record using a parameterized INSERT statement.

• **View and Search:** retrieves records with SELECT and displays them in a table; search filters by name, email, or course.

• **Update Student:** loads a selected row into the form and updates the matching student_id.

• **Delete Student:** asks for confirmation, then deletes the selected row.

• **Validation and error handling:** checks required fields, email format, semester range, phone format, duplicate email, and connection errors.

6. Procedure to Run

1. Install and start MySQL Server.

2. Install the Python connector with: py -m pip install mysql-connector-python

3. Open app.py and replace YOUR_MYSQL_PASSWORD with the local MySQL password.

4. Run database.sql in MySQL Workbench (optional because the app also creates the database and table).

5. Run the application using: py app.py

6. Test insert, select/search, update, and delete; verify each operation with SELECT * FROM students in MySQL Workbench.

7. Testing and Expected Results

Test	Expected result
Insert valid record	Record appears in app and MySQL table
Retrieve / search	Matching stored records are displayed
Update selected record	Changed values appear in app and MySQL
Delete selected record	Record no longer appears in app or table
Invalid semester / email	Validation message; invalid data is not saved
Duplicate email	Duplicate email is rejected with a message

8. Conclusion

The project demonstrates a working GUI-to-MySQL application and the core database operations required for a mini project. Records are stored persistently in MySQL, and application actions can be verified directly with SQL queries. Before submission, the student should run the application on their own computer and attach genuine screenshots of the interface and MySQL results.

Screenshot evidence to attach separately: application window, MySQL table, successful insert, successful update, and successful delete.